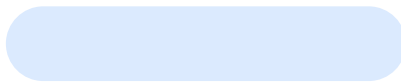


RMOne AI Agents

Technical Architecture Documentation

Embed Integration System Specification

Version 2.0 | January 2026



Document Information

Classification:	Technical - Internal Use Only
Target Audience:	Development & Engineering Teams
Last Updated:	January 28, 2026
Prepared By:	RMOne Engineering Team

Table of Contents

1.	System Overview	3
2.	Token Generation Architecture	3
3.	Authentication Flow	4
4.	Domain Restriction Mechanism	5
5.	Role-Based Access Control	6
6.	Session Management	7
7.	Security Considerations	7
8.	API Reference	8
9.	Database Schema	9
10.	Deployment Configuration	10

1. System Overview

The RMAgent AI Agents Embed System provides a secure, token-based authentication mechanism that enables external websites to integrate the AI-powered chatbot interface. The system is designed to operate within iframe contexts while maintaining strict security controls and domain-based access restrictions.

- Token Generation Service - Cryptographically secure identifier creation
- Domain Validation Layer - Origin verification and subdomain matching
- Role-Based Access Control - Granular permission management
- Session Isolation - Independent chat history per embed instance
- Token Store - In-memory authentication state management

2. Token Generation Architecture

2.1 Token Specification

Each embed link is assigned a unique 16-character alphanumeric token generated using the nanoid library with a cryptographically secure random number generator.

Token Properties:

Length	16 characters
Character Set	A-Za-z0-9 (62 characters)
Entropy	~95 bits
Collision Probability	< 1 in 10 ²⁸
Generation Method	crypto.getRandomValues()

2.2 Token Generation Code

```
import { nanoid } from 'nanoid';

// Generate cryptographically secure 16-character token
const embedId = nanoid(16);

// Example output: "jgUSnbPDCMftMgPF"
// Stored in embed_links table as primary identifier
```

2.3 Token Storage

Tokens are stored in two locations:

- Database: Permanent storage in embed_links table with metadata
- Runtime Memory: Active tokens cached in embedTokenStore Map for fast validation

3. Authentication Flow

3.1 Embed Initialization Sequence

When an external website loads the embed iframe, the following authentication sequence occurs:

- 1 Client Loads Iframe**
Parent website embeds iframe with embed ID in URL path
- 2 Origin Detection**
Embed page detects parent window origin via `document.referrer`
- 3 Token Validation**
POST `/api/embed/validate` with `embedId` and `parentOrigin`
- 4 Domain Verification**
Server validates origin against `allowed_domain` in database
- 5 Role Assignment**
Server returns role (`superadmin/admin/user`) for the embed
- 6 Session Creation**
Client stores token in `sessionStorage`, creates embed session

3.2 Token Transmission

Unlike cookie-based authentication, embed tokens are transmitted via HTTP headers to bypass

third-party cookie restrictions:

```
// Client-side: All API requests include token header
const response = await fetch('/api/query', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-Embed-Token': sessionStorage.getItem('embedToken')
  },
  body: JSON.stringify({ question: userQuery })
});
```

4. Domain Restriction Mechanism

4.1 Domain Validation Logic

The domain restriction

system provides flexible yet secure access control. When validating a request, the following checks are performed in order:

```
// Domain validation algorithm
const requestDomain = parentOrigin.toLowerCase();
const allowedDomain = embedLink.allowed_domain.toLowerCase();

const isAllowed =
  allowedDomain === '*' ||           // Wildcard
  process.env.NODE_ENV === 'development' || // Dev mode
  !parentOrigin ||                   // Direct access
  requestDomain === allowedDomain || // Exact match
  requestDomain.endsWith('.' + allowedDomain); // Subdomain
```

4.2 Matching Rules

Allowed Domain	Request Origin	Result	Rule Applied
rmone.com	rmone.com	ALLOWED	Exact Match
rmone.com	portal.rmone.com	ALLOWED	Subdomain
rmone.com	api.portal.rmone.com	ALLOWED	Nested Subdomain
rmone.com	othersite.com	DENIED	No Match
*	anysite.com	ALLOWED	Wildcard
portal.rmone.com	rmone.com	DENIED	Parent Domain

4.3 Subdomain Wildcard Behavior

Setting the allowed domain to a root domain automatically permits all subdomains. This is achieved through suffix matching:

```
// Subdomain matching implementation
const subdomainMatch = requestDomain.endsWith('.') + allowedDomain);

// Example: allowedDomain = "rmone.com"
// "portal.rmone.com".endsWith(".rmone.com") != true
// "staging.api.rmone.com".endsWith(".rmone.com") != true
// "rmone.com".endsWith(".rmone.com") != false (handled by exact match)
```

5. Role-Based Access Control

5.1 Role Hierarchy

The embed system implements a three-tier role hierarchy with progressively restrictive permissions:

SUPERADMIN

Full system access: All logs, FAQ management, chat editing, integration management

ADMIN

Elevated access: Own query logs, FAQ management, chat editing (no integration access)

USER

Basic access: Chat interface only, view own chat history, no administrative features

5.2 Permission Matrix

Feature	Superadmin	Admin	User
Chat with AI	✓	✓	✓
View Own Chat History	✓	✓	✓
Delete Own Chats	✓	✓	✓
View Query Logs	All Logs	Own Logs	—

Edit Chat Titles

Logs Tab in Results

5.3 Role Check Implementation

```
// Server-side role validation
const embedData = embedTokenStore.get(embedToken);
const isAdmin = embedData.role === 'admin' || embedData.role === 'superadmin';
const isSuperadmin = embedData.role === 'superadmin';

// Client-side permission checks
const canViewLogsTab = isAdminOrAbove;
const canManageFAQ = isAdminOrAbove;
const canEditChat = isAdminOrAbove;
```

6. Session Management

6.1 Session Isolation

Each embed instance maintains an isolated session to prevent cross-contamination of chat data:

```
// Embed user identification
const embedUserId = 'embed_' + embedId; // e.g., "embed_jgUSnbPDCMftMgPF"

// Chat ownership
chat.userId = embedUserId; // All chats scoped to this embed

// Session storage (client-side)
sessionStorage.setItem('embedToken', embedId);
sessionStorage.setItem('embedRole', role);
sessionStorage.setItem('embedUserId', embedUserId);
```

6.2 Token Store Architecture

Active embed sessions are cached in memory for fast validation:

```
// In-memory token store (server/simpleAuth.ts)
export const embedTokenStore = new Map<string, {
  embedId: string;
  role: 'superadmin' | 'admin' | 'user';
  userId: string;
  validatedAt: Date;
}>();

// Token validation populates the store
```


7. Security Considerations

7.1 Attack Mitigation

Cross-Site Request Forgery:

Token transmitted via custom header, not cookies

Token Guessing:

95-bit entropy makes brute force infeasible

Domain Spoofing:

Origin validation via `document.referrer` and `parentOrigin`

Session Hijacking:

`sessionStorage` isolated per origin, cleared on tab close

Privilege Escalation:

Role stored server-side, not modifiable by client

7.2 Best Practices

- Use specific domains instead of wildcards (*) in production
- Create separate embed links for different roles
- Regularly audit active embed links and disable unused ones
- Monitor "last_used" timestamps to identify inactive embeds

8. API Reference

8.1 Embed Validation Endpoint

POST /api/embed/validate

Validates an embed token and returns authorization status with role assignment.

Request Body:

```
{
  "embedId": "jgUSnbPDCMftMgPF",
  "parentOrigin": "https://portal.rmone.com"
}
```

Success Response (200):

```
{
  "valid": true,
  "role": "admin",
  "name": "RMOne Portal - Admin Access"
}
```

Error Response (403):

```
{
  "valid": false,
  "error": "Domain not authorized for this embed link"
}
```

8.2 Embed Link Management

POST /api/embed-links (Superadmin only)

```
// Create new embed link
{
  "name": "Client Portal - User Access",
  "allowed_domain": "client.rmone.com",
  "role": "user"
}
```

GET /api/embed-links (Superadmin only)
Returns all embed links with usage statistics.

DELETE /api/embed-links/:id (Superadmin only)

Deactivates an embed link. Existing sessions continue until page refresh.

9. Database Schema

9.1 Embed Links Table

```
-- MS SQL Server Schema
CREATE TABLE embed_links (
  id          VARCHAR(16) PRIMARY KEY, -- nanoid token
  name        NVARCHAR(255) NOT NULL,  -- Display name
  allowed_domain NVARCHAR(255) NOT NULL, -- Domain restriction
  role        VARCHAR(20) NOT NULL,     -- superadmin|admin|user
  is_active   BIT DEFAULT 1,            -- Enable/disable flag
  created_at  DATETIME2 DEFAULT GETDATE(),
  last_used   DATETIME2 NULL,           -- Usage tracking
  created_by  NVARCHAR(255) NULL        -- Creator identifier
);

-- Indexes for performance
CREATE INDEX idx_embed_active ON embed_links(is_active);
CREATE INDEX idx_embed_domain ON embed_links(allowed_domain);
```

9.2 Schema Relationships

The `embed_links` table operates independently but integrates with the chat system through the `userId` field:

```
-- Chat ownership for embed users
chats.user_id = 'embed_' + embed_links.id

-- Example: All chats for embed "jgUSnbPDCMftMgPF"
SELECT * FROM chats
WHERE user_id = 'embed_jgUSnbPDCMftMgPF';
```

9.3 Data Lifecycle

- Embed links persist until manually deleted
- Chat history retained per embed instance
- `last_used` updated on each successful validation
- Inactive embeds (`is_active = 0`) reject all requests

10. Deployment Configuration

10.1 Environment Variables

APP_MSSQL_URL	Application database connection string
CLIENT_MSSQL_URL	Client project database connection
OPENAI_API_KEY	OpenAI API key for GPT-5 queries
NODE_ENV	production development
PORT	Server port (default: 5000)

10.2 Production URLs

The embed system is deployed at:

- Primary: <https://rmchat.vyaasai.com>
- Embed Pattern: <https://rmchat.vyaasai.com/embed/{embedId}>

10.3 CORS Configuration

The server accepts requests from any origin when accessed via embed tokens. Standard session-based authentication still requires same-origin or configured CORS.

10.4 Iframe Requirements

Parent websites embedding the chatbot must ensure:

- HTTPS protocol (required for secure context)
- No X-Frame-Options: DENY headers blocking iframes
- Allow scripts from rmchat.vyaasai.com domain

End of Document